



µLab worksheet • Counting words with DataFrames and with RDDs

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • CHECK THE SPARK CLUSTER IS STILL RUNNING • STEP 1

Every command in this round is a submission to the Spark cluster built in µLab 2. Nothing here runs on your laptop's own cores, so the first thing to establish is what the cluster currently is: one master, and one core for every worker pod.

1 ☐ Count the pods, and write down how many cores that gives you.

Typed in the `kubectl` shell from µLab 2. If that shell was closed, start another one with the same `docker run` line, from the repository folder.

```
kubectl get pods
kubectl logs deploy/spark-master | grep -c "Registering worker"
```

<code>get pods</code>	List the pods. Expect one <code>spark-master</code> and however many <code>spark-worker</code> pods µLab 2 left behind, all reading 1/1 Running.
<code>grep -c "Registering worker"</code>	Count the lines in which a worker announced itself. Each worker was given <code>--cores 1</code> , so this is also the cluster's core count. It counts every registration since the master started, so a worker that was replaced is counted twice.

Set the cluster to **four** workers, so that every group is running the same experiment and the numbers below mean something when they are compared at the caucus.

```
kubectl scale deployment spark-worker --replicas=4
```

`scale ... --replicas=4` Edit one field of a Deployment that already exists. Give it fifteen seconds and re-run `kubectl get pods`.

worker pods Running _____ cores available _____

If there are no pods at all, the cluster was lost. Re-apply the manifest from µLab 2 panel 3. The image is already in the cluster's store, so this time the pods start in seconds rather than minutes.

2 • SOME DEFINITIONS

Everything this round asks you to explain can be answered from this block. The words for the cluster itself, **driver**, **executor**, **master** and **worker**, were defined in µLab 2 and are not repeated.

partition	The piece of a dataset that one core works on at a time. Splitting the data is what lets several cores work at once, so nothing is parallel until it is partitioned. Both scripts print how many partitions the read produced, under <code>input partitions</code> . That is not the same number as the <code>200</code> that appears later in the plan, which is how many the shuffle produces.
task	The work of one partition, run on one core. Spark schedules it, ships it to an executor and collects the result, and it pays that cost once per task whether the partition holds a gigabyte or four kilobytes. Partitions are therefore not free, and the number of them is the single most important thing about a Spark job.
transformation	An instruction that describes a new dataset without producing one. <code>select</code> , <code>filter</code> , <code>groupBy</code> , <code>map</code> return immediately and read nothing. They build a description, and that is all they do.
action	An instruction that demands an answer, and therefore runs everything described so far. The two in these scripts are <code>take</code> and <code>takeOrdered</code> . Until one of them is reached, no file has been opened.
shuffle	Moving rows between partitions so that rows belonging together end up together. Counting a word needs every copy of that word in one place, and they start scattered across every file, so every partition has to send something to every

	other. It is the most expensive thing a Spark job does, and on a real cluster it crosses the network. In a plan it is printed as <code>Exchange</code> .
DataFrame	Rows in named columns, described by expressions Spark can read. Because Spark can read the description, it can rewrite it before running: choose the order, choose how to group, decide how many partitions. The rewritten result is the plan .
RDD	A collection of Python objects with no columns and no types, transformed by functions Spark cannot look inside. Spark's original API, from before DataFrames were built on top of it. Because a lambda is opaque, nothing can be rewritten: what is written is what runs, in the order it was written.
plan, and lineage	The plan is what the optimiser decided; the lineage is the chain as it was written. <code>explain()</code> prints the first and <code>toDebugString()</code> prints the second. An RDD has no plan to print because there is no optimiser to print one.
stopword	A word dropped before counting because its frequency says nothing. This corpus has 203 of them, ordinary English plus the names of mail header fields. They matter twice here: they are why <code>university</code> finishes fifth, from the <code>Organization</code> : lines rather than from anyone discussing universities, and they are why one line of the plan in panel 5 is enormous.

3 • COUNT THE WORDS WITH THE DATAFRAME API • STEP 2

The script below counts every word in 18'846 Usenet messages and prints the fifteen most common. What this step is for is not the word list: it is that the **plan comes out above the words**. Nothing before the last line reads a single file, so Spark can print what it intends to do before it has done any of it.

The script, with the noise removed, is eight lines:

```
lines = spark.read.text("../20news/*.txt")           # a transformation
words = lines.select(explode(split(lower(col("value")), r"\W+"))) # a transformation
words = words.filter(length ≥ 3 and not numeric and not a stopwords)
counts = words.groupBy("word").count().orderBy(desc("count")) # still nothing ran
counts.explain()                                         # prints the plan
rows = counts.take(15)                                  # THIS one runs it
print(rows)
print(lines.rdd.getNumPartitions())
```

2 ☐ Submit it to the cluster, in the kubectl shell.

The driver runs inside the master pod, for the reason `μLab 2` panel 9 gives.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
/lab/wordcount_df.py /lab/data
```

<code>exec deploy/spark-master</code>	Run a command inside the master's pod. <code>KIND / NAME</code> again, so the pod's own name is not needed.
<code>--</code>	End of <code>kubectl</code> 's arguments. Everything after belongs to the command being run, so <code>--master</code> is read by Spark.
<code>spark-submit</code>	Spark's launcher: start a driver, ask a master for cores, run the code, print, exit.
<code>--master spark://...:7077</code>	Which cluster to ask. The Service name and submit port from <code>μLab 2</code> . Change nothing here and the same line works on a cluster of any size.
<code>/lab/wordcount_df.py</code>	The program to run. This path is inside the pods , where <code>μLab 2</code> attached the repository, and it is the same path for the driver and for every executor.
<code>/lab/data</code>	The script's own argument: which folder to read. Without it the script looks for <code>data</code> beside itself, which is not where it is from inside a pod.

Several hundred lines of Spark's own logging come first. The output that belongs to this exercise begins at the first banner line, a row of equals signs reading *the plan, before a single byte has been read*. Everything above it can be ignored.

Write these down now. They are printed once, at the end of this run, and panel 6 needs them after the next run has scrolled them away. On a four-worker cluster this took **11.7 seconds** and reported **589** input partitions, with `one` on top at 14'620.

elapsed _____ input partitions _____
top word _____ its count _____

Now answer the first question.

Which line of the script above made work actually happen? The eight-line skeleton is at the top of this panel, and panel 2 separates the two kinds of instruction. Your own output brackets the answer: everything printed above the second banner line, the one reading *now an action, and only now does anything run*, happened without reading anything.

the line _____

4 • START THE RDD COUNT, THEN LEAVE IT RUNNING • STEP 3

The second script gets the same fifteen words by different machinery: no columns, no optimiser, just functions applied one after another. It takes **minutes** rather than seconds: on a four-worker cluster it took **555 seconds**, which is nine minutes. Why is the question panel 6 asks. Start it now and go straight to panel 5. Do not sit and watch it.

3 ☐ Open a SECOND kubectl shell, and submit the other script from there.

A second docker run line, exactly the one from µLab 2, in a second terminal. The first shell keeps the DataFrame output on screen, which panel 5 needs.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
/lab/wordcount_rdd.py /lab/data
```

/lab/wordcount_rdd.py The only thing that changes. Every other argument is the one explained in panel 3.
second shell Not a second cluster: both shells talk to the same master. Two clients, one cluster, exactly as in µLab 1.

Read one number off it straight away, then walk away.

This script prints its lineage within the first few seconds, and the number in parentheses at the head of the first line is its partition count. The input partitions line at the bottom says the same thing, but only arrives when the whole job is over. Take it from the lineage now:

```
(18846) PythonRDD[6] at RDD at PythonRDD.scala:53 []
|   MapPartitionsRDD[5] at mapPartitions at PythonRDD.scala:160 []
|   |   ShuffledRDD[4] at partitionBy at <unknown>:0 []
+--(18846) PairwiseRDD[3] at reduceByKey at /lab/wordcount_rdd.py:36 []
|   PythonRDD[2] at reduceByKey at /lab/wordcount_rdd.py:36 []
|   |   /lab/data/20news/*.txt MapPartitionsRDD[1] at textFile at <unknown>:0 []
|   |   /lab/data/20news/*.txt HadoopRDD[0] at textFile at <unknown>:0 []
```

input partitions (now) _____ elapsed (when it lands) _____

5 • FIND THE SHUFFLES IN THE PLAN • STEP 4

The plan printed in step 2 is what Spark decided to do, written as a tree. Read it **bottom-up**: the bottom row happens first and the top row last. Two of its rows move rows between partitions, and those two are what this step is about.

4 ☐ Scroll back to the plan in the first shell, and find the two Exchange rows.

The copy below is the same plan with one line shortened, so that it can be marked up on paper. Circle the two rows on this sheet; check they are in your own output too.

```
AdaptiveSparkPlan isFinalPlan=false
+- Sort [count DESC NULLS LAST], true, 0
  +- Exchange rangepartitioning(count DESC NULLS LAST, 200), ENSURE_REQUIREMENTS
    +- HashAggregate(keys=[word], functions=[count(1)])
      +- Exchange hashpartitioning(word, 200), ENSURE_REQUIREMENTS
        +- HashAggregate(keys=[word], functions=[partial_count(1)])
          +- Filter ((length(word) ≥ 3) AND NOT RLIKE(word, ^[0-9]+$)
            AND NOT word INSET < 203 stopwords, ONE LINE on your screen >)
            +- Generate explode(split(lower(value), \W+, -1)), false, [word]
              +- FileScan text [value] ... InMemoryFileIndex(18846 paths)
```

The angle brackets on the eighth row are this sheet's, not Spark's: on your screen all 203 stopwords are printed there, on one line, which is why that row runs off the right of the terminal. Spark also numbers each column and each shuffle (word#3 , plan_id=26); both are internal and are dropped above.

What each row is, bottom to top.

FileScan text	Open the files and produce one row per line of text. This is the first thing that happens, and the only row that touches a disk.
InMemoryFileIndex(18846 paths)	How many files Spark found. Note it, because panel 6 compares it with the two partition counts.
Generate explode(split(...))	Cut each line into words and turn one row of text into many rows of one word. split cuts, explode is what turns one row into many.
\W+	The pattern to cut on: one or more characters that are not letters or digits. It is also why don't becomes don and t, and therefore why short words are dropped.
Filter	Throw rows away. RLIKE is a regular-expression match, here used to drop pure numbers; INSET is membership of a fixed list, here the 203 stopwords.
HashAggregate partial_count	Count within each partition, without talking to any other partition. Cheap, local, and incomplete: the same word is still counted in many places.
Exchange hashpartitioning(word, 200)	The first shuffle. Send every row to the partition chosen by a hash of its word, so that all copies of one word land together. 200 is how many partitions come out.
HashAggregate count	Now that each word is in one place, add up the partial counts. This row is the answer.
Exchange rangepartitioning(count, 200)	The second shuffle. Send rows to partitions by range of count, so that partition 1 holds the largest counts and partition 200 the smallest. Sorting needs that; counting did not.
Sort	Order within each partition. After the range shuffle, that is enough to have the whole thing in order.
AdaptiveSparkPlan isFinalPlan=false	This is the plan from before the run. Spark is allowed to change it while running, once it can see how much data there actually is, and false says it has not done so yet.

Now answer what the two shuffles are for.

The two `Exchange` rows both move every row across the cluster, and they do it for different reasons. Say, in one line each, what would go wrong if each were removed. The two glosses above name what each one groups rows by, and the row immediately above each shuffle is what needed that grouping.

One more number to be careful with: the `200` in both shuffle rows is neither 589 nor 18'846. Say which of the three the reader produced, and which the shuffle produced. Panel 2, under **partition**, distinguishes them.

Solved means panels 1 to 5 are done: the DataFrame count finished and its four numbers written down, the RDD count started and its partition count taken from the lineage, and the two `Exchange` rows found and explained. The RDD clock may still be empty here, and goes in at panel 6.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • COMPARE THE TWO PARTITION COUNTS AND THE TWO CLOCKS

Both scripts printed the same fifteen words with the same counts, on the same cluster, from the same files. Everything that differs between them is machinery. Copy the four numbers you wrote down in panels 3 and 4 into one place.

DataFrame partitions _____ DataFrame elapsed _____

RDD partitions _____ RDD elapsed _____

One fact you need and cannot work out from the output: `sc.textFile`, which the RDD script uses, gives **at least one partition per file**. The DataFrame reader packs small files together instead, until the bucket it is filling reaches a size limit.

Two questions, and both are arithmetic before they are opinion.

One of the two partition counts is exactly the number of files, which panel 5's `InMemoryFileIndex` row also printed. Divide the larger partition count by the smaller one and write the result down. Then say what that number is counting.

the quotient _____

Now the clocks. The two runs did identical arithmetic on identical data and one took far longer. Say what the extra time was spent on, using the definition of **task** in panel 2 and the two partition counts above. The answer is not that one API is written better than the other.

7 • SPLIT THE SAME JOB THREE WAYS AND TIME EACH ONE

Panel 6 compared two partition counts that the two readers happened to choose. This one chooses them on purpose. `slowdown.py` runs the same count three times over the same 34 MB, at 8 partitions, then 800, then 8000, and prints a wall clock for each. The number of distinct words it finds is printed beside each clock and it never changes, which is the point: the answer is identical all three times, so the only thing being measured is what it costs to administer the work.

Wait until the RDD count from step 3 has finished before submitting this. A standalone master gives every core it has to the first application that asks, so a second submission does not share: it waits, printing `Initial job has not accepted any resources` until the first one is over.

```
kubectrl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
/lab/slowdown.py /lab/data
```

`/lab/slowdown.py` The only argument that differs from panel 3. The script sets the partition counts itself, with `repartition(n)`, which is a transformation that says nothing except *redistribute into n pieces*.

It prints the master, the partitions the read chose, then one row per split. Record the three clocks: expect the middle to be roughly twice the first, and the last several times the middle again. Ignore the `cores Spark will use` line, which is Spark's default parallelism with a floor of two, read before the executors have reported in. Panel 1 has the cluster's real core count.

8 _____ 800 _____ 8000 _____

distinct words, all three _____

8 • SHRINK THE CLUSTER TO ONE CORE AND RUN IT AGAIN

Panel 7 varied the partitions and held the cores fixed. Now hold the partitions fixed and vary the cores. Every worker offers one core, so the cluster's size is a field in a Deployment, and changing it is the command `μLab 1` used on the whale. This is the longest run on the sheet, about six minutes on one core against two on four, so start it and read the question below while it works.

```
kubectl scale deployment spark-worker --replicas=1
kubectl get pods
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  /lab/slowdown.py /lab/data
```

`--replicas=1` One worker, one core. Wait until `kubectl get pods` shows a single `spark-worker` before submitting, or the job will start with cores that are about to disappear.
 the wait This is the longest run on the sheet: about **six minutes** on one core, against two on four. Start it, then read the question below while it works.

8 _____ 800 _____ 8000 _____

Six clocks, one sentence.

Read the six numbers as a table: cores across, partitions down. Every column rises as the partitions go up, and every row falls as the cores go up. Say in one sentence why 8000 partitions over 34 MB is slower than 8, **on both cluster sizes**. Panel 2 defines **task**, and 34 MB divided by 8000 is the size of one of them.

Put the cluster back before you leave: `kubectl scale deployment spark-worker --replicas=4`.

9 • EXTRA MILE • MAKE THE READER CHOOSE A DIFFERENT NUMBER OF PARTITIONS

589 was not a guess. The reader charges a fixed cost per file, then fills each partition until the total reaches a limit. Both are settings. Predict the new partition count before you look.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --conf spark.sql.files.maxPartitionBytes=32m \
  /lab/wordcount_df.py /lab/data
```

`--conf` Set one Spark setting for this submission only. Nothing on the cluster is changed and the next job is unaffected.
`spark.sql.files.maxPartitionBytes` How much a single input partition may hold before the reader starts another one. The default is 128m, and this asks for a quarter of that.

partitions predicted _____ partitions measured _____

elapsed _____ elapsed at the default _____

The other setting is `spark.sql.files.openCostInBytes`, the fixed charge per file, which defaults to 4m. With 18'846 files, those two numbers are enough to work out 589 exactly. Do the arithmetic, then say what the best partition count depends on: the size of the data, or the size of the cluster.

10 · ANNEX · GETTING PAST TWO THINGS THAT GO WRONG

The `Filter` row is unreadable because `explain()` prints every value of an `INSET` test, and there are 203 of them. `explain("formatted")` prints the tree first and the conditions afterwards, one per row, which fits on a screen. Using it means editing the script, which is why the core does not.

If a submission never starts. The driver waits, printing `Initial job has not accepted any resources every fifteen seconds`. It means no executor could be started, and the usual cause here is that there are no workers: panel 8 scales them to one. `kubectl get pods` settles it in one line. The other two causes are a submission queued behind one that already holds every core, which panel 7 warns about, and a driver with no name, which μ Lab 2 panel 9 covers.

Done when the referee can point at the two shuffles in the plan, and say why the same count took nine minutes one way and twelve seconds the other.